

Read this first

- The Flowstage API is designed to help artists and their teams **streamline and hyper-customize their content workflow** beyond what the platform supports in the interface (e.g. through personalized web interfaces, automations that integrate with existing systems).
- At this time, this API is a **work in progress** and will continue to evolve - please don't hesitate to reach out to the team if you experience issues or have endpoint requests.
- All documentation below assumes you know how to use **Flowstage** and are familiar with the terminology - if not, we highly recommend you check out our **docs** and **get comfortable with the workflow before attempting to automate any part of it.**
- **Content creation via the Flowstage API is bound to your account's monthly capacity. Test carefully before running massive batches of content!**



Last updated March 2026 by Rom V

Quick start

1. Get your API key

1. Log in to [Flowstage](#)
2. Navigate to **Advanced features** → **API** on the sidebar
3. Click **Create key**
4. Copy your key (starts with **fs_**)

⚠ **Important:** Your API key is shown only once. Store it securely.



2. Make your first request

```
curl -H "X-API-Key: fs_your_key_here" \  
https://api.theflowstage.com/v1/aesthetics
```

3. Complete workflow example

```
# 1. Create a new aesthetic  
curl -X POST \  
  -H "X-API-Key: fs_your_key" \  
  https://api.theflowstage.com/v1/aesthetics
```

```
-H "Content-Type: application/json" \  
-d '{"name": "My Collection"}' \  
https://api.theflowstage.com/v1/aesthetics
```

2. List your aesthetics

```
curl -H "X-API-Key: fs_your_key" \  
https://api.theflowstage.com/v1/aesthetics
```

3. Create a video edit with rendering

```
curl -X POST \  
-H "X-API-Key: fs_your_key" \  
-H "Content-Type: application/json" \  
-d '{  
  "name": "My First Edit",  
  "aesthetic_id": "aesthetic-uuid",  
  "preset_name": "preset1",  
  "audio_id": "audio-uuid",  
  "section_start_time": 0,  
  "section_end_time": 10,  
  "render": true  
' \  
https://api.theflowstage.com/v1/video-edits/draft
```

3. Poll for render completion

```
curl -H "X-API-Key: fs_your_key" \  
https://api.theflowstage.com/v1/video-edits/{edit-id}/progress
```

4. Schedule to social media (auto-schedules to next available slot)

```
curl -X POST \  
-H "X-API-Key: fs_your_key" \  
-H "Content-Type: application/json" \  
-d '{  
  "video_edit_id": "edit-uuid",  
  "caption": "Check out my video!",  
  "hashtags": ["#awesome", "#video"]  
'
```



```
}' \
```

```
https://api.theflowstage.com/v1/social-accounts/{account-id}/posts
```

API reference

Base URL: `https://api.theflowstage.com`

All endpoints require: `X-API-Key` header

Aesthetics

POST Create aesthetic

Create a new aesthetic.

POST /v1/aesthetics

Request:

```
{
  "name": "Summer vibes",
  "description": "Beach content collection"
}
```

Fields:

Field	Type	Required	Description
name	string	Yes	Name for the aesthetic
description	string	No	Description



Example:

```
curl -X POST \
  -H "X-API-Key: fs_your_key" \
  -H "Content-Type: application/json" \
  -d '{"name": "Summer vibes", "description": "Beach content"}' \
  https://api.theflowstage.com/v1/aesthetics
```

Response:

```
{
  "id": "uuid",
```

```
"name": "Summer vibes",  
"description": "Beach content",  
"created_at": "2026-03-05T18:10:00Z"  
}
```

Common errors:

```
{"detail": "Facet limit reached (5/5)"}
```



PATCH Update aesthetic

Update an aesthetic's name or description.

```
PATCH /v1/aesthetics/{aesthetic_id}
```

Request:

```
{  
  "name": "Updated name",  
  "description": "Updated description"  
}
```

Fields (all optional):

Field	Type	Description
name	string	New name for the aesthetic
description	string	New description

Example:

```
curl -X PATCH \  
  -H "X-API-Key: fs_your_key" \  
  -H "Content-Type: application/json" \  
  -d '{"name": "Winter vibes", "description": "Cold weather content"}' \  
  https://api.theflowstage.com/v1/aesthetics/{id}
```

Response:

```
{  
  "id": "uuid",  
  "name": "Winter vibes",  
  "description": "Cold weather content",  
  "updated_at": "2026-03-05T18:55:00Z"  
}
```

Common errors:

```
{"detail": "Aesthetic not found"}  
{"detail": "No fields to update"}
```

GET List aesthetics

Get all your aesthetics.



GET /v1/aesthetics

Example:

```
curl -H "X-API-Key: fs_your_key" \  
https://api.theflowstage.com/v1/aesthetics
```

Response:

```
{  
  "aesthetics": [  
    {  
      "id": "uuid",
```



```
"name": "My fanpage",  
"description": "Content for my fanpage",  
"video_preset_names": ["preset1", "preset2"],  
"created_at": "2026-01-01T00:00:00Z"  
}  
]  
}
```



GET Get aesthetic details

Get full aesthetic with all media.

```
GET /v1/aesthetics/{aesthetic_id}
```

Example:

```
curl -H "X-API-Key: fs_your_key" \  
  "https://api.theflowstage.com/v1/aesthetics/{id}?video_sample=5&audio_sample=3"
```

Query parameters:

Parameter	Type	Description
video_sample (optional)	integer	Return random N videos
photo_sample (optional)	integer	Return random N photos
audio_sample (optional)	integer	Return random N audios
hook_sample (optional)	integer	Return random N hooks

Response:



```
{
  "id": "uuid",
  "name": "My fanpage",
  "description": "Content for my fanpage",
  "videos": [
    {
      "id": "uuid",
      "name": "Clip 1",
      "url": "https://...",
      "duration": 30.5
    }
  ],
  "audios": [
    {
      "id": "uuid",
      "name": "Song Title",
      "duration": 180.0,
      "sections": [
        {
          "id": "uuid",
          "name": "Chorus",

```

```
        "start_time": 30.0,  
        "end_time": 60.0  
      }  
    ]  
  },  
  ],  
  "photos": [  
    {  
      "name": "Photo 1",  
      "url": "https://..."  
    }  
  ],  
  "hooks": [  
    {  
      "text": "Hook text"  
    }  
  ],  
  "video_preset_names": ["preset1", "preset2"],  
  "slideshow_preset_names": ["preset1", "preset2"]  
}
```



GET Aesthetic summaries

Get all aesthetics with media counts and video thumbnails in a single optimized request.

GET /v1/aesthetics/summary

Example:

```
curl -H "X-API-Key: fs_your_key" \  
https://api.theflowstage.com/v1/aesthetics/summary
```

Response:

```
{  
  "aesthetics": [  
    {  
      "id": "uuid",  
      "name": "My fanpage",  
      "description": "Content for my fanpage",  
      "created_at": "2026-01-01T00:00:00Z",  
      "video_count": 24,  
      "audio_count": 3,  
      "photo_count": 12,  
      "hook_count": 8,  
      "video_thumbnails": [  
        "https://thumbnail1.jpg",  
        "https://thumbnail2.jpg"  
      ]  
    }  
  ]  
}
```



Use case: Dashboard views that need media counts and visual previews without loading full media details.

Performance: Single query with joins - returns all aesthetics with counts and up to 8 video thumbnails per aesthetic in 2 database queries total.

GET Batch get aesthetics



Get multiple aesthetics efficiently in one request. Returns only **id** , **name** , and **audios** (with sections) for each aesthetic.

```
GET /v1/aesthetics/batch?ids={comma_separated_ids}
```

Query parameters:

Parameter	Type	Required	Description
ids	string	Yes	Comma-separated aesthetic IDs

Example:

```
curl -H "X-API-Key: fs_your_key" \
  "https://api.theflowstage.com/v1/aesthetics/batch?ids=uuid1,uuid2,uuid3"
```

Response:

```
{
  "aesthetics": [
    {
      "id": "uuid1",
      "name": "My fanpage",
      "audios": [
        {
          "id": "uuid",
          "name": "Song Title",
          "duration": 180.0,
          "sections": [
            {
              "id": "uuid",
              "name": "Chorus",
              "start_time": 30.0,
              "end_time": 60.0
            }
          ]
        }
      ]
    }
  ]
}
```



Use case: Efficiently fetch aesthetic details for multiple accounts' bound aesthetics in one request instead of making N individual requests.

Performance: Optimized with bulk queries - fetches all aesthetics, audios, and

[API documentation](#)[Back to home](#)[Log in](#)

> Read this first

✓ Quick start

1. Get your API key
2. Make your first request
3. Complete workflow example

✓ API reference

- Aesthetics
- Video edits
- Social accounts
- Posts & scheduling
- Hook batch operations

> Scheduling system

> Code examples

> Error handling

> Rate limits & quotas

> Troubleshooting

GET Get aesthetic videos

Get all videos with analysis data.

```
GET /v1/aesthetics/{aesthetic_id}/videos
```

Example:

```
curl -H "X-API-Key: fs_your_key" \
  https://api.theflowstage.com/v1/aesthetics/{id}/videos
```

Response:

```
{
  "videos": [
    {
      "id": "uuid",
      "name": "Clip 1",
      "url": "https://...",
      "duration": 30.5,
```



Need help integrating?

Contact us →

```
"analysis": {  
  "tags": ["outdoor", "sunset", "beach"],  
  "scene_summary": "A person walking on the beach at sunset",  
  "scene_focus": "person",  
  "location": "beach",  
  "time_of_day": "sunset"  
}  
}  
]  
}
```



DELETE Remove video from aesthetic

Remove a video from your aesthetic - this does not delete the video itself!

```
DELETE /v1/aesthetics/{aesthetic_id}/videos/{video_id}
```

Example:

```
curl -X DELETE \  
  -H "X-API-Key: fs_your_key" \  
  https://api.theflowstage.com/v1/aesthetics/{id}/videos/{video_id}
```


Response:

- **204 No Content** - Removed successfully
- **404 Not Found** - Video not linked to this aesthetic

GET Get aesthetic audios

Get all audios with sections and lyrics.



```
GET /v1/aesthetics/{aesthetic_id}/audios
```

Example:

```
curl -H "X-API-Key: fs_your_key" \
https://api.theflowstage.com/v1/aesthetics/{id}/audios
```

Response:

```
{
  "audios": [
    {
      "id": "uuid",
```

```
"name": "Song Title",
"duration": 180.0,
"sections": [
  {
    "id": "uuid",
    "name": "Chorus",
    "start_time": 30.0,
    "end_time": 60.0,
    "lines": [
      {
        "text": "Lyric line",
        "start_time": 0.0,
        "end_time": 2.5
      }
    ]
  }
]
```



POST Add audio to aesthetic

Add an existing audio to your aesthetic - this does not create or analyze the audio (you must do this in the Flowstage interface!)

POST /v1/aesthetics/{aesthetic_id}/audios

Request:

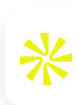
```
{  
  "audio_id": "uuid"  
}
```

Example:

```
curl -X POST \  
  -H "X-API-Key: fs_your_key" \  
  -H "Content-Type: application/json" \  
  -d '{"audio_id": "uuid"}' \  
  https://api.theflowstage.com/v1/aesthetics/{id}/audios
```

Response:

```
{  
  "success": true,  
  "audio_id": "uuid",  
  "audio_name": "Song title",  
  "message": "Audio added to aesthetic"  
}
```

Common errors:

```
{"detail": "Audio not found"}  
{"detail": "Audio is already linked to this aesthetic"}
```

DELETE Remove audio from aesthetic

Remove an audio from your aesthetic - this does not delete the audio!



```
DELETE /v1/aesthetics/{aesthetic_id}/audios/{audio_id}
```

Example:

```
curl -X DELETE \  
  -H "X-API-Key: fs_your_key" \  
  https://api.theflowstage.com/v1/aesthetics/{id}/audios/{audio_id}
```

Response:

- **204 No Content** - Removed successfully
- **404 Not Found** - Audio not linked to this aesthetic

GET Get video usage stats

See which videos are used most in your edits.

```
GET /v1/aesthetics/{aesthetic_id}/video-usage?limit=100
```

Example:

```
curl -H "X-API-Key: fs_your_key" \
  "https://api.theflowstage.com/v1/aesthetics/{id}/video-usage?limit=100"
```

Query parameters:

Parameter	Type	Description
limit (optional)	integer	Max edits to check (default: 100, range: 1-500)

Response:

```
{
  "videos": [
    {
      "video_id": "uuid",
```

```
    "use_count": 12,  
    "last_used_at": "2026-01-15T10:30:00Z",  
    "positions": [0, 1, 2],  
    "edits_used_in": ["edit1", "edit2"]  
  }  
],  
  "total_edits_checked": 42  
}
```



POST Add a new video to an aesthetic

Add a video from an external URL to your aesthetic.

POST /v1/aesthetics/{aesthetic_id}/videos

Request:

```
{  
  "url": "https://videos.pexels.com/...",  
  "name": "Stock Video",  
  "duration": 30.0,  
  "thumbnail_url": "https://...",  
  "analyze": false  
}
```

Requirements:

Requirement	Description
Public accessibility	URL must be publicly accessible
CORS support	URL must support CORS
Format	Video must be in a supported format (MP4, WebM)

Response:

```
{
  "id": "uuid",
  "name": "Stock Video",
  "url": "https://...",
  "analysis_triggered": false
}
```



Video edits

POST Create video edit (draft endpoint)

Create a new video edit with intelligent routing. The endpoint automatically routes to the appropriate creation mode based on your parameters.

POST /v1/video-edits/draft

Two creation modes:

1. **Draft mode** (manual video selection) - Provide **videos** array
2. **Auto-draft mode** (use Flowstage's video selection) - Omit **videos** array



Draft mode (with videos)

Manually select which videos to use in your edit.

Request:

```
{
  "name": "My Edit",
  "aesthetic_id": "uuid",
  "preset_name": "preset1",
  "videos": [
    {
      "video_id": "uuid",
      "start_time": 0.0,
      "end_time": 10.0
    }
  ],
  "audio_id": "uuid",
```



```
"section_start_time": 30.0,  
"section_end_time": 40.0,  
"hook": "Optional text overlay",  
"render": true  
}
```

Fields:

Field	Type	Description
name	string	Display name for your edit
aesthetic_id	string	Parent aesthetic UUID
preset_name	string	Styling preset (get from aesthetic details)
videos	array	Required for draft mode - Array of video clips with timing
audio_id	string	Background audio UUID
section_start_time	number	Audio section start (seconds)
section_end_time	number	Audio section end (seconds)
hook (optional)	string	Optional text overlay
render (optional)	boolean	Trigger rendering (default: true)



Auto-draft mode (Flowstage selects videos)

Let Flowstage automatically select videos from your aesthetic based on the hook and audio.

Request:

```
{
  "name": "Flowstage drafted edit",
  "aesthetic_id": "uuid",
  "preset_name": "preset1",
  "hook": "it's just a song",
  "audio_id": "uuid",
  "section_start_time": 30.0,
  "section_end_time": 40.0,
  "render": false
}
```



Fields:

Field	Type	Description
name (optional)	string	Display name (auto-generated if omitted)
aesthetic_id	string	Parent aesthetic UUID
preset_name (optional)	string	Styling preset (random if omitted)
hook	string	Required for auto-draft - Text overlay

Flowstage		
Field	Type	Description
audio_id	string	Background audio UUID
section_start_time	number	Audio section start (seconds)
section_end_time	number	Audio section end (seconds)
render (optional)	boolean	Trigger rendering (default: false)

Note: Omit the **videos** array to trigger auto-draft mode. Flowstage will select appropriate videos from your aesthetic, minimizing reuse and maximizing variety.



Flipbook mode (auto-draft only)

Create a flipbook-style edit with photos instead of videos. Only available in auto-draft mode.

Request:

```
{
  "name": "Photo flipbook",
  "aesthetic_id": "uuid",
  "hook": "it's just a song",
  "audio_id": "uuid",
  "section_start_time": 0.0,
  "section_end_time": 10.0,
  "is_flipbook": true,
```

```
"loop_interval_seconds": 0.5,  
"render": false  
}
```

Additional fields:

Field	Type	Description
is_flipbook	boolean	Enable flipbook mode (photos instead of videos)
loop_interval_seconds (optional)	number	Time per photo (default: 0.3)



Important:

- Flipbook mode automatically selects up to 8 random photos from your aesthetic
- Returns **422 Unprocessable Entity** if aesthetic has no photos
- Cannot use **videos** and **is_flipbook** together

Response

```
{  
  "video_edit_id": "uuid",  
}
```

```
"status": "idea",  
"message": "Video edit created successfully"  
}
```

Monthly limits:

- Number of edits per month depends on your plan
- API returns **429 Too Many Requests** when limit exceeded



Examples

```
# Draft mode: Manual video selection  
curl -X POST \  
  -H "X-API-Key: fs_your_key" \  
  -H "Content-Type: application/json" \  
  -d '{  
    "name": "My edit",  
    "aesthetic_id": "uuid",  
    "preset_name": "preset1",  
    "videos": [{"video_id": "uuid", "start_time": 0, "end_time": 10}],  
    "audio_id": "uuid",  
    "section_start_time": 0,  
    "section_end_time": 10,  
    "render": true  
  }' \  
https://api.theflowstage.com/v1/video-edits/draft
```

Auto-draft mode: Flowstage selects videos

```
curl -X POST \  
  -H "X-API-Key: fs_your_key" \  
  -H "Content-Type: application/json" \  
  -d '{  
    "aesthetic_id": "uuid",  
    "hook": "it's just a song",  
    "audio_id": "uuid",  
    "section_start_time": 0,  
    "section_end_time": 10,  
    "render": false  
  }' \  
https://api.theflowstage.com/v1/video-edits/draft
```

Flipbook mode: Flowstage selects photos

```
curl -X POST \  
  -H "X-API-Key: fs_your_key" \  
  -H "Content-Type: application/json" \  
  -d '{  
    "aesthetic_id": "uuid",  
    "hook": "it's just a song",  
    "audio_id": "uuid",  
    "section_start_time": 0,  
    "section_end_time": 10,  
    "is_flipbook": true,  
    "loop_interval_seconds": 0.5,  
    "render": false  
  }' \  
https://api.theflowstage.com/v1/video-edits/draft
```



GET Get video edit

Get video edit details and render status.

```
GET /v1/video-edits/{edit_id}
```

Example:

```
curl -H "X-API-Key: fs_your_key" \
  https://api.theflowstage.com/v1/video-edits/{edit_id}
```

Response:

```
{
  "id": "uuid",
  "status": "approved",
  "render_status": "done",
  "render_progress": 1.0,
  "render_url": "https://...",
  "duration": 30.0,
  "created_at": "2026-01-01T00:00:00Z"
}
```

Status values:



Value	Description
draft	Not yet approved
approved	Ready for use/scheduling
archived	Soft deleted

Render status values:

Value	Description
pending	Not started
processing	In progress (check render_progress 0.0-1.0)
done	Completed successfully
error	Failed (check error message)



GET Poll render progress

Check if rendering is complete. **Poll this endpoint** after creating an edit with **render: true** .

GET /v1/video-edits/{edit_id}/progress

Response:


```
{
  "edit_id": "uuid",
  "status": "done",
  "progress": 1.0,
  "url": "https://...",
  "message": "Render complete",
  "error": null
}
```

Polling strategy:

```
import time

def wait_for_render(edit_id):
    max_attempts = 60 # 5 minutes
    for i in range(max_attempts):
        response = requests.get(
            f"{BASE_URL}/v1/video-edits/{edit_id}/progress",
            headers={"X-API-Key": API_KEY}
        )
        data = response.json()

        if data["status"] == "done":
            return data["url"]
        elif data["status"] == "error":
            raise Exception(f"Render failed: {data.get('error')}")

        time.sleep(5) # Poll every 5 seconds

    raise Exception("Render timeout")
```



GET Download video edit

Get redirect to rendered video file.

```
GET /v1/video-edits/{edit_id}/download
```

Response:

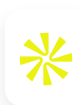
- **307 Temporary Redirect** - Redirects to S3 download URL
- **404 Not Found** - Video not rendered yet

Usage:

```
# Download with cURL (follows redirect)
curl -L -H "X-API-Key: fs_your_key" \
  "https://api.theflowstage.com/v1/video-edits/{id}/download" \
  -o video.mp4
```

Social accounts

GET List social accounts



Get all your connected social media accounts.

```
GET /v1/social-accounts
```

Example:

```
curl -H "X-API-Key: fs_your_key" \  
https://api.theflowstage.com/v1/social-accounts
```



Response:

```
{  
  "accounts": [  
    {  
      "id": "uuid",  
      "platform": "tiktok",  
      "handle": "@myaccount",  
      "timezone": "America/New_York",  
      "default_timeslots": [8, 12, 20],  
      "bound_aesthetic_id": "uuid"  
    }  
  ]  
}
```

Account fields:

Field	Type	Description
platform	string	tiktok , instagram , or youtube
handle	string	Display name (e.g., @username)
timezone	string	IANA timezone (e.g., America/New_York , Europe/London)
default_timeslots	array	Array of hours (0-23) for posting
bound_aesthetic_id	string	Linked aesthetic for content



Posts & scheduling

GET List account posts

Get posts for a social account.

GET /v1/social-accounts/{account_id}/posts

Query parameters:

Parameter	Type	Description
start_date (optional)	string	Filter by date range (YYYY-MM-DD)
end_date (optional)	string	Filter by date range (YYYY-MM-DD)
include_posted (optional)	boolean	Include already posted (default: false)
status (optional)	string	Filter by status (e.g., planned , sent)

Example:



```
curl -H "X-API-Key: fs_your_key" \
  "https://api.theflowstage.com/v1/social-accounts/{id}/posts?start_date=2026-02-
```

Response:

```
{
  "posts": [
    {
      "id": "uuid",
      "social_account_id": "uuid",
      "video_edit_id": "uuid",
      "slideshow_edit_id": null,
      "caption": "Post caption",
      "hashtags": ["#tag1", "#tag2"],
      "time_scheduled": "2026-02-22T14:00:00Z",
      "time_posted": "2026-02-22T14:05:00Z",
      "is_posted": true,
```

```
{
  "status": "posted",
  "tt_autopublish": false,
  "tt_publish_id": null,
  "created_at": "2026-02-01T10:00:00Z",
  "views": 12500,
  "likes": 890,
  "comments": 45,
  "shares": 23,
  "metrics_updated_at": "2026-02-23T08:00:00Z",
  "hook": "Check this out!"
}
```



Metrics fields:

- **views** , **likes** , **comments** , **shares** - Performance metrics (null if not yet available)
- **metrics_updated_at** - When metrics were last refreshed from the platform
- **time_posted** - Actual time the post was published (null if not yet posted)
- **slideshow_edit_id** - ID of slideshow edit if applicable (null for video posts)
- **hook** - Hook text extracted from edit_proposal (null if no hook or external post)

Hook extraction: The hook is automatically extracted from the video/slideshow edit's **edit_proposal** based on text overlays that:

- Have a non-null **target_position**
- Have a **style** containing "hook" (case-insensitive)

This matches the logic used in the Insights page for creative analysis.

GET **Batch get posts**

Get posts for multiple social accounts efficiently in one request.

```
GET /v1/posts/batch?account_ids={comma_separated_ids}&start_date={date}&end_date={date}
```



Query parameters:

Parameter	Type	Required	Description
account_ids	string	Yes	Comma-separated account IDs (max 20)
start_date	string	Yes	Start date (ISO8601 format)
end_date	string	Yes	End date (ISO8601 format)
include_posted	boolean	No	Include already posted (default: false)
status	string	No	Filter by status

Limits:

- Max 20 accounts per request

- Max 90-day date range
- Returns all posts in range

Example:

```
curl -H "X-API-Key: fs_your_key" \
  "https://api.theflowstage.com/v1/posts/batch?account_ids=uuid1,uuid2,uuid3&star
```



Response:

```
{
  "posts_by_account": {
    "uuid1": [
      {
        "id": "uuid",
        "social_account_id": "uuid1",
        "video_edit_id": "uuid",
        "slideshow_edit_id": null,
        "caption": "Post caption",
        "hashtags": ["#tag1"],
        "time_scheduled": "2026-03-03T14:00:00Z",
        "time_posted": "2026-03-03T14:05:00Z",
        "is_posted": true,
        "status": "posted",
        "tt_autopublish": false,
        "tt_publish_id": null,
        "created_at": "2026-03-01T10:00:00Z",
        "views": 8200,
        "likes": 456,
```



```
    "comments": 28,  
    "shares": 12,  
    "metrics_updated_at": "2026-03-04T09:00:00Z",  
    "hook": "You need to see this"  
  }  
],  
  "uuid2": [],  
  "uuid3": [...]  
}  
}
```



Metrics fields:

- **views , likes , comments , shares** - Performance metrics (null if not yet available)
- **metrics_updated_at** - When metrics were last refreshed from the platform
- **time_posted** - Actual time the post was published (null if not yet posted)
- **slideshow_edit_id** - ID of slideshow edit if applicable (null for video posts)
- **hook** - Hook text extracted from edit_proposal (null if no hook or external post)

Use case: Efficiently fetch posts for multiple accounts (e.g., dashboard calendar view) in one request instead of making N individual requests.

Performance: Optimized with bulk queries - verifies account ownership, fetches posts, and extracts hooks in 4 total database queries regardless of how many posts returned.

GET Video edit metadata

Get comprehensive metadata for video edits including videos used, audio info, and optional analyses.

```
GET /v1/video-edits/metadata?edit_ids={comma_separated_ids}&include_analyses={boo
```

Query parameters:



Parameter	Type	Required	Description
edit_ids	string	Yes	Comma-separated edit IDs (max 20)
include_analyses	boolean	No	Include video analyses (default: false, expensive)

What it returns:

- Hook (extracted from edit_proposal)
- Preset name and video type
- Audio track info (name, section title, timing)
- Videos used in sequence with clip timings
- Optional: Video analyses (tags, scene summary)

Limits:

- Max 20 edits per request

- **include_analyses=false** by default (analyses are expensive)

Example:

```
curl -H "X-API-Key: fs_your_key" \
  "https://api.theflowstage.com/v1/video-edits/metadata?edit_ids=uuid1,uuid2&incl
```

Response:



```
{
  "edits": {
    "uuid1": {
      "id": "uuid1",
      "name": "My Video Edit",
      "preset_name": "POV",
      "video_type": "pov",
      "hook": "Check this out!",
      "audio": {
        "id": "audio-uuid",
        "name": "Trending Sound 2024",
        "duration": 180.5,
        "section_title": "Chorus",
        "section_start_time": 45.2,
        "section_end_time": 60.8
      },
    },
    "videos": [
      {
        "id": "video-uuid-1",
        "name": "Clip 1",
        "url": "https://...",

```

```
"thumbnail": "https://...",
"duration": 30.5,
"clip_start_time": 5.0,
"clip_end_time": 10.0,
"order_index": 0,
"analysis": {
  "tags": ["outdoor", "sunset", "beach"],
  "scene_summary": "Person walking on beach at sunset",
  "scene_focus": "wide shot",
  "location": "beach",
  "time_of_day": "sunset"
},
{
  "id": "video-uuid-2",
  "name": "Clip 2",
  "url": "https://...",
  "thumbnail": "https://...",
  "duration": 25.0,
  "clip_start_time": 0.0,
  "clip_end_time": 5.5,
  "order_index": 1,
  "analysis": { ... }
},
"created_at": "2026-03-01T10:00:00Z"
},
"uuid2": { ... }
}
```



Use case: Deep creative analysis - get all media used in edits with their analyses to identify patterns (e.g., which video types perform best, common themes in top posts).

Performance:

- Without analyses: ~200ms for 20 edits
- With analyses: ~800ms for 20 edits (expensive due to media_analyses lookup)

Recommended workflow:

1. **GET /v1/posts/batch** → Get posts with metrics and hooks
2. Identify top performing posts
3. **GET /v1/video-edits/metadata** → Get detailed creative data for those edits
4. Analyze patterns (hooks, video types, audio sections, visual themes)



POST Create post (schedule)

Schedule a video edit to a social account.

`POST /v1/social-accounts/{account_id}/posts`

Request:

```
{
  "video_edit_id": "uuid",
  "date": "2026-02-22",
  "slot_hour": 14,
  "caption": "Check out my video!",
  "hashtags": ["#awesome", "#video"],
  "tt_autopublish": false
}
```



Scheduling modes:

date	slot_hour	Behavior
✓	✓	Schedule to exact date and hour
✓	×	Pick first open slot on that day
×	✓	Find nearest future day with that hour available
×	×	Auto-schedule to nearest available timeslot

Examples:

```
# Auto-schedule (recommended)
curl -X POST \
  -H "X-API-Key: fs_your_key" \
  -H "Content-Type: application/json" \
  -d '{
    "video_edit_id": "uuid",
```

```
"caption": "My post",  
"hashtags": ["#video"]  
}' \  
https://api.theflowstage.com/v1/social-accounts/{id}/posts
```

Schedule to specific date and time

```
curl -X POST \  
-H "X-API-Key: fs_your_key" \  
-H "Content-Type: application/json" \  
-d '{  
  "video_edit_id": "uuid",  
  "date": "2026-02-22",  
  "slot_hour": 14,  
  "caption": "My post"  
}' \  
https://api.theflowstage.com/v1/social-accounts/{id}/posts
```



Validation rules:

Rule	Description
Status	Video edit must have status: "approved"
Rendering	Video edit must be rendered (has render_url)
Timeslot	slot_hour must be in account's default_timeslots
Availability	Slot must not be occupied
Timing	Date/time must be in the future (account timezone)
Daily limit	Max 4 posts per day

Rule	Description
Weekly limit	Same video can't be scheduled twice in same week on same account

Response:

```
{
  "id": "uuid",
  "social_account_id": "uuid",
  "video_edit_id": "uuid",
  "time_scheduled": "2026-02-22T19:00:00Z",
  "caption": "Check out my video!",
  "hashtags": ["#awesome", "#video"],
  "status": "planned",
  "created_at": "2026-01-01T00:00:00Z"
}
```



Common errors:

```
{"detail": "slot_hour 3 is not in account's timeslots: [8, 12, 20]}"
{"detail": "Slot 14:00 on 2026-02-22 is already occupied"}
{"detail": "Maximum 4 posts per day reached for 2026-02-22"}
{"detail": "This video edit is already scheduled this week on this account"}
{"detail": "Cannot schedule in the past"}
{"detail": "Video edit must be rendered before scheduling"}
```


GET Get post

Get single post with video edit details.

```
GET /v1/posts/{post_id}
```

Example:

```
curl -H "X-API-Key: fs_your_key" \  
https://api.theflowstage.com/v1/posts/{post_id}
```

Response:

```
{  
  "id": "uuid",  
  "social_account_id": "uuid",  
  "video_edit_id": "uuid",  
  "caption": "Post caption",  
  "hashtags": ["#tag1"],  
  "time_scheduled": "2026-02-22T19:00:00Z",  
  "is_posted": false,  
  "status": "planned",  
  "tt_autopublish": false,  
  "tt_publish_id": null,  
  "created_at": "2026-02-01T10:00:00Z",  
  "video_edit": {  
    "id": "uuid",
```



```
"name": "Edit name",  
"render_url": "https://...",  
"render_progress": 1.0  
}  
}
```

PATCH Update post

Update post caption, hashtags, or reschedule.

```
PATCH /v1/posts/{post_id}
```

Example:

```
curl -X PATCH \  
  -H "X-API-Key: fs_your_key" \  
  -H "Content-Type: application/json" \  
  -d '{  
    "caption": "Updated caption",  
    "hashtags": ["#new", "#tags"]  
  }' \  
  https://api.theflowstage.com/v1/posts/{post_id}
```



Request:

```
{  
  "caption": "Updated caption",  
  "hashtags": ["#new", "#tags"],  
  "date": "2026-02-23",  
  "slot_hour": 12  
}
```

Rules:

Rule	Description
Lock after send	Cannot update if already sent to TikTok
Validation	Rescheduling follows same validation as create
Fields	All fields are optional

DELETE Delete post

Delete a scheduled post.

DELETE /v1/posts/{post_id}



Example:

```
curl -X DELETE \  
  -H "X-API-Key: fs_your_key" \  
  https://api.theflowstage.com/v1/posts/{post_id}
```

Response:

- **204 No Content** - Deleted successfully
- **404 Not Found** - Post not found
- **409 Conflict** - Cannot delete (already posted)



GET Get account schedule

Get week view of schedule with slot availability.

```
GET /v1/social-accounts/{account_id}/schedule?week_start=2026-02-17
```

Example:

```
curl -H "X-API-Key: fs_your_key" \
  "https://api.theflowstage.com/v1/social-accounts/{id}/schedule?week_start=2026-
```

Query parameters:

Parameter	Type	Description
week_start	string	Monday of week to view (YYYY-MM-DD)



Response:

```
{
  "account_id": "uuid",
  "week_start": "2026-02-17",
  "timezone": "America/New_York",
  "default_timeslots": [8, 12, 20],
  "days": [
    {
      "date": "2026-02-17",
      "slots": [
        {
          "hour": 8,
          "post_id": null,
          "is_occupied": false
        },
        {
          "hour": 12,
          "post_id": "uuid",
          "is_occupied": true
        }
      ]
    }
  ]
}
```

```
}  
]  
}  
]  
}
```

Hook batch operations



POST Batch add hooks

Add multiple hooks to an aesthetic in a single transaction.

```
POST /v1/aesthetics/{aesthetic_id}/hooks/batch-add
```

Example:

```
curl -X POST \  
  -H "X-API-Key: fs_your_key" \  
  -H "Content-Type: application/json" \  
  -d '{"hook_ids": ["uuid1", "uuid2", "uuid3"]}' \  
  https://api.theflowstage.com/v1/aesthetics/{id}/hooks/batch-add
```

Request:

```
{  
  "hook_ids": ["uuid1", "uuid2", "uuid3"]  
}
```

Response:

```
{  
  "success": true,  
  "added": 3,  
  "skipped": 0,  
  "message": "Added 3 hook(s), skipped 0 duplicate(s)"  
}
```

**POST** Batch remove hooks

Remove multiple hooks from an aesthetic in a single transaction.

POST /v1/aesthetics/{aesthetic_id}/hooks/batch-remove

Example:

```
curl -X POST \  
  -H "X-API-Key: fs_your_key" \  
  -H "Content-Type: application/json" \  
  -d '{"hook_ids": ["uuid1", "uuid2"]}' \  
  https://api.theflowstage.com/v1/aesthetics/{id}/hooks/batch-remove
```

Request:

```
{  
  "hook_ids": ["uuid1", "uuid2"]  
}
```

Response:

```
{  
  "success": true,  
  "removed": 2,  
  "message": "Removed 2 hook(s) from aesthetic"  
}
```



POST Replace all hooks

Replace all hooks in an aesthetic with a new set. Pass an empty array to remove all hooks.

```
POST /v1/aesthetics/{aesthetic_id}/hooks/replace-all
```

Example:



```
curl -X POST \
  -H "X-API-Key: fs_your_key" \
  -H "Content-Type: application/json" \
  -d '{"hook_ids": ["uuid1", "uuid2"]}' \
  https://api.theflowstage.com/v1/aesthetics/{id}/hooks/replace-all
```

Request:

```
{
  "hook_ids": ["uuid1", "uuid2"]
}
```

Response:

```
{  
  "success": true,  
  "removed": 5,  
  "added": 2,  
  "message": "Replaced hooks: removed 5, added 2"  
}
```



Scheduling system

How scheduling works

Flowstage uses **timezone-aware scheduling** with **timeslots** to ensure consistent posting times.

Timezones

Each social account has a timezone (e.g., **America/New_York** , **Europe/London**). All scheduling is done in the account's local time.

Example:

```
Account timezone: America/New_York (EST, UTC-5)
Input: date="2026-02-22", slot_hour=14
Interpreted as: 2026-02-22 2:00 PM EST
Stored as: 2026-02-22T19:00:00Z (UTC)
```

Timeslots



Timeslots are configured posting times for each account (default: **[8, 12, 20]** = 8 AM, 12 PM, 8 PM).

Rules:

Rule	Description
Slot capacity	Each timeslot can hold 1 post per day
Daily limit	Max 4 posts per day (hard limit)
Valid hours	slot_hour must match one of the account's timeslots

Example:

```
{
  "timezone": "America/New_York",
```

```
"default_timeslots": [9, 14, 18]
}
```

This account can post at 9 AM, 2 PM, and 6 PM EST.

Auto-scheduling



When you don't provide **date** or **slot_hour**, the API automatically finds the next available timeslot:

1. Checks today's remaining slots
2. If all occupied, checks tomorrow
3. Continues up to 30 days ahead
4. Returns error if no slots available

Example:

```
# Auto-schedule (easiest)
response = requests.post(
    f"{BASE_URL}/v1/social-accounts/{account_id}/posts",
    headers={"X-API-Key": API_KEY},
    json={
        "video_edit_id": edit_id,
        "caption": "My post"
    }
)
```

```
)  
# API finds next available slot automatically
```

Duplicate prevention

The same video edit cannot be scheduled twice in the same week on the same account.



Week definition: Monday-Sunday containing the scheduled date.

This prevents accidentally posting the same content multiple times.

Code examples

Complete Python integration

```
import requests  
import time  
from datetime import datetime, timedelta  
  
API_KEY = "fs_your_key"  
BASE_URL = "https://api.theflowstage.com"
```

```
class FlowstageClient:
    def __init__(self, api_key):
        self.api_key = api_key
        self.base_url = BASE_URL
        self.headers = {"X-API-Key": api_key}

    def list_aesthetics(self):
        """Get all aesthetics"""
        response = requests.get(
            f"{self.base_url}/v1/aesthetics",
            headers=self.headers
        )
        response.raise_for_status()
        return response.json()["aesthetics"]

    def get_aesthetic(self, aesthetic_id):
        """Get aesthetic with all media"""
        response = requests.get(
            f"{self.base_url}/v1/aesthetics/{aesthetic_id}",
            headers=self.headers
        )
        response.raise_for_status()
        return response.json()

    def create_video_edit(self, aesthetic_id, name, videos, audio_id,
                          section_start, section_end, hook=None, render=True):
        """Create a video edit"""
        payload = {
            "name": name,
            "aesthetic_id": aesthetic_id,
            "preset_name": "preset1", # Get from aesthetic details
            "videos": videos,
            "audio_id": audio_id,
            "section_start_time": section_start,
            "section_end_time": section_end,
```



```
        "hook": hook,
        "render": render
    }

    response = requests.post(
        f"{self.base_url}/v1/video-edits",
        headers=self.headers,
        json=payload
    )
    response.raise_for_status()
    return response.json()

def wait_for_render(self, edit_id, max_wait=300, poll_interval=5):
    """Poll for render completion"""
    start_time = time.time()

    while time.time() - start_time < max_wait:
        response = requests.get(
            f"{self.base_url}/v1/video-edits/{edit_id}/progress",
            headers=self.headers
        )
        response.raise_for_status()
        data = response.json()

        if data["status"] == "done":
            return data["url"]
        elif data["status"] == "error":
            raise Exception(f"Render failed: {data.get('error')}")

        print(f"Rendering... {data['progress']*100:.0f}%")
        time.sleep(poll_interval)

    raise Exception("Render timeout")

def schedule_post(self, account_id, video_edit_id, caption,
```



```
        hashtags=None, date=None, slot_hour=None):
    """Schedule a post (auto-schedules if date/hour not provided)"""
    payload = {
        "video_edit_id": video_edit_id,
        "caption": caption,
        "hashtags": hashtags or []
    }

    if date:
        payload["date"] = date
    if slot_hour is not None:
        payload["slot_hour"] = slot_hour

    response = requests.post(
        f"{self.base_url}/v1/social-accounts/{account_id}/posts",
        headers=self.headers,
        json=payload
    )
    response.raise_for_status()
    return response.json()
```

Usage

```
client = FlowstageClient(API_KEY)
```

1. Get aesthetic

```
aesthetics = client.list_aesthetics()
aesthetic = client.get_aesthetic(aesthetics[0]["id"])
```

2. Create video edit

```
video_id = aesthetic["videos"][0]["id"]
audio = aesthetic["audios"][0]
section = audio["sections"][0]
```

```
edit = client.create_video_edit(
    aesthetic_id=aesthetic["id"],
```




```
name="API Generated Edit",
videos=[{
    "video_id": video_id,
    "start_time": 0,
    "end_time": 10
}],
audio_id=audio["id"],
section_start=section["start_time"],
section_end=section["end_time"],
hook="Check this out!",
render=True
)

# 3. Wait for render
render_url = client.wait_for_render(edit["video_edit_id"])
print(f"Rendered: {render_url}")

# 4. Schedule to social media
accounts = requests.get(
    f"{BASE_URL}/v1/social-accounts",
    headers={"X-API-Key": API_KEY}
).json()["accounts"]

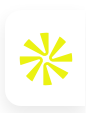
post = client.schedule_post(
    account_id=accounts[0]["id"],
    video_edit_id=edit["video_edit_id"],
    caption="Created with Flowstage API!",
    hashtags=["#api", "#automation"]
)

print(f"Scheduled for: {post['time_scheduled']}")
```



JavaScript/TypeScript integration

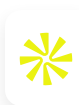
```
interface FlowstageConfig {  
  apiKey: string;  
  baseUrl?: string;  
}  
  
class FlowstageClient {  
  private apiKey: string;  
  private baseUrl: string;  
  
  constructor(config: FlowstageConfig) {  
    this.apiKey = config.apiKey;  
    this.baseUrl = config.baseUrl || 'https://api.theflowstage.com';  
  }  
  
  private async request(endpoint: string, options: RequestInit = {}) {  
    const response = await fetch(`${this.baseUrl}${endpoint}`, {  
      ...options,  
      headers: {  
        'X-API-Key': this.apiKey,  
        'Content-Type': 'application/json',  
        ...options.headers,  
      },  
    });  
  
    if (!response.ok) {  
      const error = await response.json();  
      throw new Error(error.detail || 'API request failed');  
    }  
  
    return response.json();  
  }  
}
```



```
async listAesthetics() {
  const data = await this.request('/v1/aesthetics');
  return data.aesthetics;
}

async createVideoEdit(params: {
  name: string;
  aestheticId: string;
  videos: Array<{ video_id: string; start_time: number; end_time: number }>;
  audioId: string;
  sectionStart: number;
  sectionEnd: number;
  hook?: string;
  render?: boolean;
}) {
  return this.request('/v1/video-edits', {
    method: 'POST',
    body: JSON.stringify({
      name: params.name,
      aesthetic_id: params.aestheticId,
      preset_name: 'preset1',
      videos: params.videos,
      audio_id: params.audioId,
      section_start_time: params.sectionStart,
      section_end_time: params.sectionEnd,
      hook: params.hook,
      render: params.render ?? true,
    }),
  });
}

async waitForRender(editId: string, maxWait = 300000): Promise<string> {
  const startTime = Date.now();
  const pollInterval = 5000;
```



```

while (Date.now() - startTime < maxWait) {
  const progress = await this.request(`/v1/video-edits/${editId}/progress`);

  if (progress.status === 'done') {
    return progress.url;
  } else if (progress.status === 'error') {
    throw new Error(`Render failed: ${progress.error}`);
  }

  console.log(`Rendering... ${((progress.progress * 100).toFixed(0))}%`);
  await new Promise(resolve => setTimeout(resolve, pollInterval));
}

throw new Error('Render timeout');
}

async schedulePost(params: {
  accountId: string;
  videoEditId: string;
  caption: string;
  hashtags?: string[];
  date?: string;
  slotHour?: number;
}) {
  return this.request(`/v1/social-accounts/${params.accountId}/posts`, {
    method: 'POST',
    body: JSON.stringify({
      video_edit_id: params.videoEditId,
      caption: params.caption,
      hashtags: params.hashtags || [],
      date: params.date,
      slot_hour: params.slotHour,
    }),
  });
}

```



```
}

// Usage
const client = new FlowstageClient({
  apiKey: process.env.FLOWSTAGE_API_KEY!
});

async function main() {
  // Create and schedule video
  const aesthetics = await client.listAesthetics();

  const edit = await client.createVideoEdit({
    name: 'API Generated',
    aestheticId: aesthetics[0].id,
    videos: [{ video_id: 'uuid', start_time: 0, end_time: 10 }],
    audioId: 'uuid',
    sectionStart: 0,
    sectionEnd: 10,
  });

  const renderUrl = await client.waitForRender(edit.video_edit_id);
  console.log('Rendered:', renderUrl);

  const post = await client.schedulePost({
    accountId: 'account-uuid',
    videoEditId: edit.video_edit_id,
    caption: 'Created with API!',
    hashtags: ['#api'],
  });

  console.log('Scheduled for:', post.time_scheduled);
}

main();
```



Error handling

HTTP status codes

Code	Meaning	Action
200	Success	Request completed
204	No Content	Delete successful
307	Redirect	Follow redirect (download)
401	Unauthorized	Check API key
404	Not Found	Resource doesn't exist
409	Conflict	Cannot modify (e.g., already posted)
422	Validation Error	Fix request data
429	Rate Limited	Monthly limit exceeded
500	Server Error	Retry with backoff
503	Service Unavailable	Dependent service down, retry



Error response format

All errors return JSON with a **detail** field:

```
{  
  "detail": "Human-readable error message"  
}
```

Common errors

Authentication:

```
{"detail": "Invalid or missing API key"}
```

Validation:

```
{"detail": "Video edit must be rendered before scheduling"}  
{"detail": "slot_hour 3 is not in account's timeslots: [8, 12, 20]"}  
{"detail": "Cannot schedule in the past"}  
{"detail": "Invalid date format. Use YYYY-MM-DD"}
```

Limits:

```
{"detail": "Monthly video edit limit exceeded"}  
{"detail": "Maximum 4 posts per day reached for 2026-02-22"}
```



```
{"detail": "All 3 timeslots are full on 2026-02-22"}
```

Conflicts:

```
{"detail": "Slot 14:00 on 2026-02-22 is already occupied"}
```

```
{"detail": "This video edit is already scheduled this week on this account"}
```



Error handling best practices

```
import requests
from requests.exceptions import HTTPError

try:
    response = requests.post(url, headers=headers, json=data)
    response.raise_for_status()
    return response.json()

except HTTPError as e:
    if e.response.status_code == 401:
        # Invalid API key
        print("Authentication failed. Check your API key.")
    elif e.response.status_code == 422:
        # Validation error
        error = e.response.json()
        print(f"Validation error: {error['detail']}")
```



```
elif e.response.status_code == 429:
    # Rate limited
    print("Monthly limit exceeded. Upgrade your plan.")
elif e.response.status_code >= 500:
    # Server error - retry
    print("Server error. Retrying...")
    time.sleep(5)
    # Retry logic here
else:
    print(f"Error: {e.response.json()['detail']}")
```



Rate limits & quotas

Monthly limits

Your plan determines monthly limits for:

- **Video edits created** - Number of new edits per month
- **Posts scheduled** - Number of posts per month
- **API requests** - Total API calls per month (future)

When exceeded, you'll receive:

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
```

```
{  
  "detail": "Monthly video edit limit exceeded"  
}
```

Daily limits

- **Max 4 posts per day** per social account (hard limit)
- **Max posts per day** also limited by number of timeslots



Best practices

- **Check limits** before bulk operations
- **Handle 429 errors** gracefully
- **Cache responses** when possible
- **Use webhooks** instead of polling (when available)

Troubleshooting

"Invalid or missing API key"

Cause: API key not in header or incorrect format

Solution:

 Correct

```
curl -H "X-API-Key: fs_..." https://...
```

 Wrong

```
curl -H "Authorization: Bearer fs_..." https://...
```



"Video edit must be rendered before scheduling"

Cause: Trying to schedule an edit that hasn't finished rendering

Solution: Poll `/v1/video-edits/{id}/progress` until `status === "done"`

"slot_hour X is not in account's timeslots"

Cause: Provided hour doesn't match account's configured timeslots

Solution:

1. Get account details: **GET /v1/social-accounts**
 2. Check **default_timeslots** field
 3. Use one of those hours, or omit **slot_hour** to auto-schedule
-

"Slot 14:00 on 2026-02-22 is already occupied"

Cause: Another post is already scheduled for that exact slot

Solution:

- Use auto-scheduling (omit **date** and **slot_hour**)
 - Pick a different hour from **default_timeslots**
 - Pick a different day
-

"Maximum 4 posts per day reached"

Cause: Hard limit of 4 posts per day per account

Solution:

- Schedule to a different day
- Delete an existing post for that day



- Use a different social account
-

"This video edit is already scheduled this week"

Cause: Duplicate prevention - same video can't be scheduled twice in one week

Solution:

- Wait until next week
 - Use a different video edit
 - Schedule to a different account
-



Render timeout

Cause: Render taking longer than expected

Solution:

- Increase `max_wait` in your polling logic
 - Try shorter videos (< 30 seconds)
 - Check backend-render service status
-

"Monthly limit exceeded"

Cause: You've hit your plan's monthly quota

Solution:

- Upgrade your Flowstage plan
- Wait until next month
- Delete unused edits/posts to free up quota

